

# Bachelor Thesis

Nsak as Framework for Scenario Based Network Security

|                 |                                                    |
|-----------------|----------------------------------------------------|
| Course of study | Bachelor of Science in Computer Science            |
| Author          | Frank Gauss (gausf1) and Lukas von Allmen (vonal3) |
| Advisor         | Wenger Hansjürg                                    |

Version 1.0 of March 26, 2026

# Abstract

One-paragraph summary of the entire study – typically no more than 250 words in length (and in many cases it is well shorter than that), the Abstract provides an overview of the study.

# Contents

# 1 Introduction

Computer networks are constantly exposed to evolving cyber threats ...

The *Network Swiss Army Knife (NSAK)* is a framework designed to execute orchestrated security scenarios consisting of reusable attack drills in a controlled network environment. The framework aims to support automated security testing and adversary emulation on network edge devices.

The development of the NSAK framework originates from previous project work, in which the core architecture and the basic scenario execution model were developed. This thesis builds upon this foundation and extends the framework by investigating how AI agents can support automated red and blue team scenarios.

...

## 1.1 Motivation

**Hypothesis:** AI can enhance red and blue team simulations in a network environment by generating and executing scenarios within the NSAK framework and assisting in the evaluation of detection results.

## 1.2 Predecessor Project: “Project 2”

## 1.3 Project Goals

For a goal to be considered **S.M.A.R.T.**, it must fulfill the following criteria:

- ▶ **Specific:** What do we want to achieve?
- ▶ **Measurable:** What are the success criteria?
- ▶ **Achievable:** How do we plan to achieve this goal?
- ▶ **Relevant:** Why is it important to achieve this goal?

Describe what we did in project 2 and what project 2 is (BFH module, possible predecessor to bachelor thesis)

- ▶ **Time-bound:** Goals categorized as *must* have to be achieved within the thesis, while *should* or *could* goals are considered optional or nice to have.

### 1.3.1 Research Goals

#### Compare Frameworks (must)

To identify the USP (unique selling point) of the NSAK framework, we will analyze and compare existing security emulation frameworks. First, relevant frameworks such as MITRE Caldera and Atomic Red Team will be identified and reviewed. Based on this analysis, the identified concepts will be compared with the architecture of the NSAK framework. The comparison will focus on how scenarios and drills are defined, how configuration and parameter handling are structured, how reusable components are organized, how automation is achieved, and which AI capabilities they provide.

The analysis and comparison will be based on the following properties:

- ▶ Concepts
- ▶ Modularity
- ▶ Configurability
- ▶ Automation
- ▶ AI Capabilities

#### Success criteria:

- ▶ At least two security emulation frameworks are identified and reviewed
- ▶ A matrix comparing the specified properties of the selected frameworks is created
- ▶ The USP of the NSAK framework is identified

#### AI Integration (must)

The goal of this research is to investigate how AI (artificial intelligence) can support automated security testing and adversary emulation. The research evaluates potential integration points where AI could enhance the scenario execution, automation, decision-making, and analysis capabilities of the NSAK framework.

#### Success criteria:

- ▶ How is AI currently used in cybersecurity automation?
- ▶ Which tasks in adversary emulation can be supported or automated by AI?
- ▶ How could AI interact with the NSAK scenarios and drills (e.g., Agents, MCP)?
- ▶ What architectural changes would be required to integrate AI into NSAK?

### 1.3.2 Framework

#### Configuration Management (must)

The goal is to extend the NSAK framework, so that scenarios and drills can expose configurable parameters to increase the flexibility of scenarios and reusability of drills. Parameters can be provided via command-line arguments or YAML files and are resolved by the framework into a unified run configuration used for scenario execution.

#### Success criteria:

- ▶ Scenarios and drills can expose configurable parameters
- ▶ The available parameters can be listed by the CLI with the `--help` option
- ▶ Parameters can be provided via CLI arguments or YAML files
- ▶ The framework resolves the inputs into a run configuration (internal data structure).

#### REST API (TBD)

Adding a REST API to the NSAK framework was already proposed in the predecessor project, as it greatly enhances interoperability with other tools. The priority of this goal depends heavily on the outcome of the goal 1.3.1 and on whether we want to fulfill the goal ??, which is prioritized as “could”. If the evaluated AI integration requires a REST API (e.g., MCP), this goal needs to be prioritized as “must”; it will be prioritized as “could”.

#### Success criteria:

- ▶ A REST API specification with feature parity to the CLI is created.

- ▶ The REST API is implemented according to the specification.
- ▶ The API is documented (Open API) and usable by external clients.
- ▶ The REST API remains optional and does not replace the CLI-based interaction.
- ▶ Authentication and Authorization are explicitly out of scope.

#### MCP Server (TBD)

The MCP Protocol allows the AI to interact with applications in a standardized way - and could therefore be necessary for implementing AI-based scenarios. If the outcome of the research task 1.3.1 highlights the necessity to specify/expose an MCP-Server to interact seamlessly with the NSAK framework, the implementation is a (must), otherwise a (could).

- ▶ A MCP-Server is specified and implemented
- ▶ Documentation how to connect to the MCP is provided.
- ▶ A form of authentication is provided.

#### Web GUI (could)

The goal is to design and implement a web-based graphical user interface that allows users to interact with the NSAK framework in a user-friendly way. The Web GUI will enable management of scenarios and drills, triggering scenario execution, and visualization of execution results and system status.

#### Success criteria:

- ▶ A simple Web GUI that interacts with the REST API has been implemented.
- ▶ The Web GUI displays a list of available scenarios and their states.
- ▶ The scenario's results can be viewed on the Web GUI
- ▶ Scenarios can be configured and executed via the Web GUI
- ▶ Authentication and authorization are explicitly out of scope.

### 1.3.3 Scenarios

#### AI – Purple Team (must)

The goal is to implement artificial intelligence according to the result of 1.3.1 and to explore the use of AI to support red, blue, and purple team exercises in the NSAK framework. The AI will be evaluated for its potential to assist in executing attacks in a scenario and analyzing detection results. The findings will help assess how AI could enhance and coordinate red and blue team activities across a network via the NSAK device.

#### Success criteria:

- ▶ The AI is integrated into NSAK as a scenario or as part of the framework, depending on the result of 1.3.1.
- ▶ The AI can execute CLI or Python tools to directly or indirectly interact with the attached network.
- ▶ An operator can interact with or stop the running AI.
- ▶ The result artifacts are stored and can be presented or reused for further scenarios.

#### Reproducible Test Environment (must)

The goal is to design and implement a reproducible and safe test environment for the NSAK framework that enables consistent execution and evaluation of scenarios and drills. The environment should provide a stable basis for testing and comparison of manual and AI-assisted scenarios.

#### Success criteria:

- ▶ The test environment can be set up reproducibly using a defined configuration or setup procedure with Infrastructure as code (e.g., Ansible)
- ▶ NSAK scenarios and drills can be executed consistently within the environment.
- ▶ The environment supports both manual and AI-assisted scenarios.
- ▶ Scenario results can be collected and analyzed for evaluation purposes.

### Network Discovery – Red Team (must)

The goal is to implement a reference red team scenario for network discovery within the NSAK framework. The scenario should simulate reconnaissance activities to identify hosts, services, and network characteristics, and the results should be comparable to the AI-Agent approach.

#### Success criteria:

- ▶ A red team network discovery scenario is implemented in the NSAK framework.
- ▶ Discovered network information is collected as artifacts.
- ▶ The scenario can be executed reproducibly within the test environment.
- ▶ The results can be compared with those generated by the AI-assisted approach.

### Intrusion Detection – Blue Team (should)

The goal is to define and implement a blue team reference scenario for intrusion detection within the NSAK framework. The scenario should allow observation and evaluation of suspicious network activity to assess detection capabilities in the test environment and compare the results with those of the AI approach.

#### Success criteria:

- ▶ A blue-team intrusion-detection scenario is implemented in the NSAK framework.
- ▶ Detection events or alerts can be observed and documented during scenario execution.
- ▶ Detection results can be compared with results from AI-assisted scenario execution.

### 1.3.4 Evaluation Goals

#### AI Scenario Evaluation (must)

The objective is to determine the effectiveness of the AI-based scenario introduced by the goal ?? within the network environment defined by the goal ?? The evaluation will assess whether the AI scenario successfully executed the expected red and blue team activities for the provided prompts and the quality of the results.

##### Success criteria:

- ▶ A methodical approach for the assessment is evaluated and documented.
- ▶ The results of the AI scenario for red and blue team activity are assessed.
- ▶ There is a conclusion on the effectiveness of the AI scenario for red and blue team activities.

#### AI vs Engineered Scenarios (must/should)

The goal of this evaluation is to compare the quality of AI-based red (network reconnaissance) (must) and blue (network intrusion detection) (should) team scenarios with that of manually engineered scenarios.

The comparison focuses on criteria such as execution reliability, reproducibility, and coverage. The results will help determine the potential benefits and limitations of AI-supported scenarios and provide insights into how AI could enhance future versions of the NSAK framework.

##### Success criteria:

- ▶ A methodical approach for the comparison is evaluated and documented.
- ▶ The results of the AI scenario are compared against the respective results of the red team scenario (must)
- ▶ The results of the AI scenario are compared against the respective results of the blue team scenario (should)
- ▶ There is a conclusion on the effectiveness of the AI scenario compared to manually engineered scenarios.

### 1.3.5 Out of Scope / Optional Feature

At the project management meeting on 05.03.26 (see Table ?? in Appendix A.7), the initial project scope was discussed. During planning, the team proposed using AI-based scenarios and comparing them with the implemented versions, which required reassessing certain milestones.

Running AI-based scenarios does not require a REST API or GUI. A reproducible test setup and further research are needed to properly evaluate this approach.

## 2 Related Research

### 2.1 Framework Comparison

Previous research, such as ‘SOK: A Survey of Open-Source Threat Emulators’, compares multiple types of threat emulation frameworks using both quantitative and qualitative metrics [?]. Among the evaluated frameworks, Atomic Red Team, Mitre Caldera, and Metasploit received high scores in categories as environments and operator, environment configuration and scenario execution.

In the following section, the selected frameworks are briefly introduced, and the unique selling point of the Network Swiss Army Knife (NSAK) is highlighted. As a point of reference, the cybersecurity community increasingly relies on the concepts of MITRE ATT&CK, which has established itself as a standard framework in the field since its introduction in 2014 [?, ?].

The MITRE ATT&CK is a globally accessible knowledge base of adversary tactics and techniques based on real-world observations [?]. The knowledge base is structured into TTP (Tactics, Techniques, and Procedures), which are organized into 14 attack behavior groups. The TTP is getting continuously updated and extended, which helps to defend against the capabilities of threat actors.

The evaluation of the proposed frameworks is based on information obtained from official documentation [?, ?, ?] and the findings of Zilberman et al. [?].

| Characteristic           | Metasploit Framework                                             | Atomic Team                                                | Red                  | MITRE Caldera                                 |
|--------------------------|------------------------------------------------------------------|------------------------------------------------------------|----------------------|-----------------------------------------------|
| Primary Purpose          | Exploitation and penetration testing framework                   | Security testing through atomic tests                      | testing small ATT&CK | Automated adversary emulation platform        |
| Relation to MITRE ATT&CK | Partial mapping via modules                                      | Direct mapping to ATT&CK techniques                        |                      | Built around ATT&CK adversary emulation       |
| Automation Level         | Medium (scripts and modules)                                     | Low (manual execution of tests)                            |                      | High (automated attack chains)                |
| Main Functionality       | Exploit development, payload delivery, post-exploitation         | Execute ATT&CK technique tests                             | small tech-          | Simulate adversary campaigns across hosts     |
| Architecture             | Modular framework with exploits, payloads, and auxiliary modules | Collection of scripts/tests organized by ATT&CK techniques |                      | Client-server platform                        |
| Typical Use Case         | Penetration testing and vulnerability validation                 | Detection validation and security control testing          |                      | Red team automation and purple team exercises |
| Difficulty Level         | Medium-High                                                      | Low-Medium                                                 |                      | Medium-High                                   |

Table 2.1: Comparison of Metasploit, Atomic Red Team, and MITRE Caldera ([?, ?, ?, ?])

As shown in Table ??, the key characteristics of the frameworks are differ greatly. **Atomic Red Team** provides libraries of attack scripts that implement individual Tactics, Techniques, and Procedures of MITRE ATT&CK. Each attack can be executed independently or chained together into a larger attack scenario. **Mitre Caldera** is a complex, agent based adversary simulation tool, whereas **Metasploit** is a framework that provides exploits, payloads and auxiliary modules. Further details of the frameworks are presented in the following sections. ??????

Table  
format-  
ting

### 2.1.1 Atomic Red Team

The Atomic Red Team repository provides documentation and configuration files for each technique. For every TTP, a YAML Ain't Markup Language (YAML) configuration file defines the execution parameters, while an enhanced Markdown file contains a human-readable description of the attack. This documentation includes a short description of the technique, execution instructions, and additional contextual information [?].

The framework can be executed directly from the command line and provides features such as logging, detailed error messages, and automatic prerequisite checks. If a technique cannot be executed, for example due to operating system constraints, the framework reports the issue and can optionally attempt to install the required prerequisites [?].

*Zilberman et al.* recommends deeper cybersecurity knowledge to use Atomic Red Team framework effectively [?]. The Open Source mentality, expandability and easily usable scripts can be a great asset for an independent framework such as the NSAK.

### 2.1.2 Caldera

Mitre Caldera is also built on the MITRE ATT&CK knowledge base and functions as a command-and-control center. A central server controls agents running on the target system, executes attacks called abilities, and sends the operation results back to the server. Every Ability is a single attack step built on the Tactics, Techniques, and Procedures. The framework ships with a graphical user interface and is primarily used for red team attack simulation, purple teaming, and detection engineering. The additional features, like a training plugin for educational purposes, include facts that can be configured as dynamic safe values, which can be stored and parsed in later steps [?].

With its provided interface and functionality, Caldera is most suitable as a threat emulator, offering a wide range of built-in features, including lateral movement and Command and Control (CNC) communication [?].

### 2.1.3 Metasploit

Metasploit is a widely used penetration testing framework for testing or attacking systems and exploiting known vulnerabilities. Metasploit follows a sequence of steps to identify a vulnerability in a target system, select a vulnerability for the attack, select an exploit that uses this vulnerability, and deliver a payload that runs on the target system. The configuration via the Command Line Interface

(CLI) requires knowledge of the target network and system and requires partial configuration [?, ?].

According to Zilberman et al., an operator requires extensive cybersecurity knowledge to effectively use the framework. To support the operator, the GUI “Armitage”, which is not further evaluated in this work, can be used [?].

#### 2.1.4 NSAK

The NSAK framework, as an independent universal Network Swiss Army Knife, is not confined and can leverage, if needed, different sets of attack frameworks. An approach could be to use an Artificial Intelligence (AI)-Enhanced Network Discovery Scenario, which provides the necessary information to configure a Metasploit attack that leads to an attack operation in Caldera and an atomic red team detection script. Hardware-based isolation and network position provide flexibility and should help reduce the operational costs of red and blue teams by reducing simple configuration overhead [?].

### 2.2 AI-Research

#### 2.2.1 AI and Language Models

Language Model (LM) describes the technique for advancing language intelligence in machines and its development history can be divided into several stages. The first stage, Statistical Language Model (SLM) emerged around the 1990s, and the basic idea is to assume, that such a model can predict the next word. But the fact that an exponential number of transition probabilities are needed, caused a lack of accuracy and led limited success.

To overcome these limitations, Neural Language Model (NLM) was introduced to learn a distributed representation of words and capture more complex patterns, leading to a major breakthrough. Building on this, the advancement of pre-trained language models Pre-trained Language model (PLM), which are first trained on large datasets and then fine-tuned for specific tasks, represented another significant development.

Building on this development, Large Language Model (LLM)s further increases the scale of PLMs by expanding parameter count and computational power [?].

In large language models, parameters are the trainable numerical values that define how the model processes input and generates output. During training, these parameters are continually adjusted to enable the model to learn patterns and

language context [?]. The models most of us use today, such as ChatGPT, Claude and Gemini are pre-trained LLMs used for language inference.

These advances led to Model Context Protocol (MCP), a protocol that allows tools and services to expose an interface specifically for AI. Further, the introduction of AI-agents is enabling a new level of autonomy for achieving concrete goals with LLMs. The combination of these technologies is now resulting in systems, where LLMs empower autonomous agents capable of decision-making to execute complex tasks.

### 2.2.2 MCP

Anthropic, a US software company focused on AI research, introduced the protocol in 2024. The MCP was introduced as an open source standard that enables AI assistants to connect to external systems, including databases, business tools, and development environments [?]. Dynamic tool discovery and schema negotiation is helping clients and agents to access these functions in a standardized manner and allows autonomous interaction. Additionally, MCP allows information to flow in both directions, so the model can send requests, and tools can send updates or events.

This architectural approach shifts the AI integration from static toward an interoperable ecosystem of AI-enabled services. On the other hand, the flexible design and open source approach of MCP unlocks multiple novel and systematic security risks that are in need of enhanced security and governance [?].

### 2.2.3 Agentic-AI

Agentic Artificial Intelligence (Agentic-AI) can be understood as an advanced machine intelligence that is capable of perceiving, reasoning, and acting with a certain level of autonomy. Traditional systems and agents are more limited to executing predefined commands, whereas Agentic-AI is not restricted in this way. Due to their ability to adapt in real time, they can continuously refine their knowledge, techniques, and data. The ability to access multiple inputs, from natural language processing to sensor data, allows Artificial Intelligence Agent (AI Agent)s to develop a good understanding of their environment [?].

Multi Agent System (MAS) defines a way in which multiple specialist agent coordinate and cooperate with each other to solve a task, that would not be solvable to one agent alone.

Therefore, the agents need a form of interaction that allows them to act reactively, often through a shared mental model, which helps create a common understanding of the complex tasks to be solved. To implement a MAS efficiently, modern

design principles such as encapsulation and modularity are important, which are helping to balance resource usage of the system and LLM [?]

Huang et al. provide evidence that Agentic-AI and MAS are capable of solving complex tasks across various sectors, including security and banking. However, these systems also introduce new potential vulnerabilities, both within AI Agent systems themselves and through their interactions with external environments.

The identified vulnerabilities can be categorized into two types: accidental and deliberate. Accidental vulnerabilities include software bugs, logical errors introduced by agents, hardware malfunctions, and poor data quality. Deliberate attacks, on the other hand, encompass adversarial attacks targeting LLMs, data poisoning, and model theft.

To mitigate these risks, a comprehensive framework consisting of governance mechanisms, proactive monitoring, regulatory compliance, and rigorous testing is required to ensure the secure and reliable operation of agentic systems [?].

### Fazit MCP and Agentic AI

In the cybersecurity podcast Risky Business, it is mentioned that MCP is “dead”. The open-source strategy of Anthropic (??) has led to the development of multiple MCP implementations that were not designed with a strong focus on security, but rather on rapid adoption of a novel technique.

This provocative claim is based on the observation that, in order to handle the excessive MCP tooling, AI Agents may instead rely on direct access to a root shell, as it provides a more efficient way of interaction [?].

### Evaluated Frameworks and MCP

Caldera uses a MCP plugin as a bridge between the model and the Representational State Transfer Application Programming Interface (REST-API), allowing the agent to plan and issue commands in a manner similar to a human operator.

Caldera introduces three main research areas. Firstly, a planner determines the order and selection of abilities and can dynamically construct new adversaries based on user requirements through prompt-based interaction.

Secondly, an adversary factory model is used to generate adversaries tailored to the operator’s specific use cases.

Thirdly, a forward-deployed agent is enhanced with Retrieval Augmented Generation (RAG) for Cyber Threat Intelligence (CTI), leveraging Structured Threat

How and why is this relevant for our thesis? how could we implement it? To which findings should we pay attention? What can we ignore for now?

Information Expression (STIX)-based data to create reproducible and testable adversaries [?].

What about the other frameworks? What about AI Agents? What can we learn from their approach? How does it affect our strategy?

## 3 Concepts

## 4 Methodology

## 5 Implementation

## 5.1 Configuration Management Implementation

### 5.1.1 Device Configuration Management

#### Create New Device Resource Type

In the context of “Project 2”[1.2](#) this resource type was already defined conceptionally and the BananaPI R4 was added as an example in the library under `lib/devices/bananapi_r4`. But the resource type was not actually added to the NSAK core and be able to manage device configurations we need to implement the Device resource type first. Analog to the other resource types, a yaml specification for defining devices in the library and multiple classes for representing and managing devices in the core are required. The listing [??](#) shows the initial specification for a device resource, which will get extended later on with a section for configuration management.

---

```
1 device: <str:resource-version>
2
3 metadata:
4   id: <str:device-id>
5   name: <str:device-name>
6   description: <str:device-description>
7   author: <str:author-email>
8   repository: <str:resource-repository>
```

---

Listing 1: Device Resource YAML Specification

In the following table [??](#) the required Python classes are listed and described. The classes will be located under `src/nsak/core/device` within their respective files.

In “Project 2”[1.2](#) it became already clear, that the described classes share a lot of boilerplate code between each other. For this reason the base classes described in [??](#) were added to get rid of the duplicated logic, which leads to better compliance with the Don’t Repeat Yourself principles. The classes will be located under `src/nsak/core/resource` within their respective files.

To complete the integration of the device resource, the new functionality must be exposed via CLI. The available bash commands are shown in the following listing [??](#).

| Class         | File Name         | Description                                                                                                                  |
|---------------|-------------------|------------------------------------------------------------------------------------------------------------------------------|
| Device        | device.py         | A dataclass, which represents a device resource as objects during runtime (usually quite similar to the yaml specification). |
| DeviceLoader  | device_loader.py  | This class returns Device objects and is used by the DeviceManager to find and load device resources from the library.       |
| DeviceManager | device_manager.py | The manager class implements the business logic and exposes an API, which can be used by the CLI.                            |

Table 5.1: Device Resource Classes

| Class           | File Name           | Description                                                                   |
|-----------------|---------------------|-------------------------------------------------------------------------------|
| Resource        | resource.py         | Base class for all resources: Scenario, Drill, Environment and Device.        |
| ResourceLoader  | resource_loader.py  | Base class which handles searching and loading resources from the filesystem. |
| ResourceManager | resource_manager.py | Base class which defines common methods for all resource manager classes.     |

Table 5.2: Resource Python Classes

```

1  # Show usage, options and subcommands for the device resource
2  nsak device --list
3
4  # List all devices found in the resource library
5  nsak device list
6
7  # Output:
8  > nsak device list
9  ID          NAME          PATH
10 bananapi_r4  BananaPI R4  <repository>/lib/devices/bananapi_r4

```

Listing 2: NSAK CLI: List Devices

## Device Configuration Management Implementation

To allow a device to expose its configuration options, the YAML specification needs to be extended accordingly. For now, the focus is only on network configuration as this is the most important information for the scenarios and drills, but the specification was done in such a way that other sections can be added later on. The following excerpt ?? shows the specification for the configuration section of the device resource YAML.

---

```
1  device: <str:resource-version>
2
3  metadata:
4      ...
5
6  configuration:
7      network:
8          ethernets:
9              <str:intreface-name>:
10                 addresses:
11                     <str:ip-address-with-subnet>:
12                         is_target: <boolean>
13                         is_management: <boolean>
```

---

Listing 3: Device Resource Configuration YAML Specification

The following code snippet located under `src/nsak/core/device/device_loader.py` ?? shows how the YAML configuration gets parsed to the respective datastructures defined in `src/nsak/core/device/device.py` ??.

---

```

1  class DeviceLoader(ResourceLoader[Device]):
2      """
3      Finds and loads devices from the library paths.
4      """
5
6      ResourceClass = Device
7
8      @classmethod
9      def _parse_device_configuration(
10         cls, data: dict[str, Any]
11     ) -> DeviceConfiguration | None:
12         if "configuration" not in data:
13             return None
14
15         ethernets = data["configuration"].get("network",
16         ↪ {}).get("ethernets", {})
17
18         return DeviceConfiguration(
19             network=NetworkConfiguration(
20                 ethernets={
21                     interface: EthernetConfiguration(
22                         name=interface,
23                         addresses={
24                             ip: IPConfiguration(
25                                 ip=ip_interface(ip),
26
27                                 ↪ is_target=address.get("is_target",
28                                 ↪ False),
29
30                                 ↪ is_management=address.get("is_management",
31                                 ↪ False),
32
33                             )
34                             for ip, address in
35                             ↪ ethernet.get("addresses",
36                             ↪ {}).items()
37
38                         },
39
40                     ),
41
42                 for interface, ethernet in ethernets.items()
43
44             )
45
46         )
47
48     @classmethod
49     def _to_resource(cls, data: dict[str, Any], path: Path) ->
50     ↪ Device:
51         """
52         Creates a Device object from a dict containing the
53         ↪ device's metadata.
54         """
55
56         return cls.ResourceClass(
57             id=str(data["metadata"]["id"]),
58             name=str(data["metadata"]["name"]),
59             path=path,
60             author=str(data["metadata"]["author"])

```

```
1 @dataclasses.dataclass(frozen=True, kw_only=True)
2 class IPConfiguration:
3     """
4     Represents an ip configuration on an interface.
5     """
6
7     ip: IPInterface
8     is_target: bool
9     is_management: bool
10
11
12 @dataclasses.dataclass(frozen=True, kw_only=True)
13 class EthernetConfiguration:
14     """
15     Represents an ethernet node.
16     """
17
18     name: str
19     addresses: dict[str, IPConfiguration]
20
21
22 @dataclasses.dataclass(frozen=True, kw_only=True, eq=True)
23 class NetworkConfiguration:
24     """
25     Represents the networking part of the configuration, heavily
26     ↪ inspired by netplan.
27     """
28
29     ethernets: dict[str, EthernetConfiguration]
30
31
32 @dataclasses.dataclass(frozen=True, kw_only=True, eq=True)
33 class DeviceConfiguration:
34     """
35     Represents the device configuration.
36     """
37
38     network: NetworkConfiguration
39
40
41 @dataclasses.dataclass(frozen=True, kw_only=True, eq=True)
42 class Device(Resource):
43     """
44     Represents a device.
45     """
46
47     id: str
48     name: str
49     path: Path
50     author: str
51     repository: str
52     configuration: DeviceConfiguration | None = None
```

In the following listing shows the newly available CLI commands for managing device configurations ??:

---

```
1 # List all subcommands for the Device resource
2 nsak device --help
3
4 # List all available devices
5 nsak device list
6
7 # Show the device details and configuration
8 nsak device show <str:device-id>
9
10 # Load a device and its configuration
11 nsak device load <str:device-id>
12
13 # Show the currently loaded device
14 nsak device loaded
15
16 # Reset the loaded device
17 nsak device unload
```

---

Listing 6: Device Configuration CLI Management

### 5.1.2 Drill Configuration Management

### 5.1.3 Scenario Configuration Management

### 5.1.4 Configuration via CLI

### 5.1.5 Configuration via CLI --config\_file

### 5.1.6 Configuration via interactive CLI Dialog

## 6 Evaluation

## 7 Discussion

## 8 Conclusion

## Bibliography

- [1] Polina Zilberman, Rami Puzis, Sanders Bruskin, Shai Shwarz, and Yuval Elovici. Sok: A survey of open-source threat emulators. *arXiv preprint*, 2020.
- [2] Blake E. Strom, Andy Applebaum, Doug Miller, Adam Nickels, Cody Pennington, and Chris Thomas. Mitre att&ck: Design and philosophy. Technical report, MITRE Corporation, 2020.
- [3] Bader Al-Sada, Alireza Sadighian, and Gabriele Oligeri. Mitre att&ck: State of the art and way forward. *ACM Computing Surveys*, 2023.
- [4] MITRE Corporation. Mitre caldera documentation, 2024.
- [5] Red Canary. Atomic red team, 2024.
- [6] Rapid7. Metasploit framework documentation, 2024.
- [7] David Kennedy, Jim O’Gorman, Devon Kearns, and Mati Aharoni. *Metasploit: The Penetration Tester’s Guide*. No Starch Press, 2011.
- [8] Frank Gauss and Lukas von Allmen. Nsak (network swiss army knife). <https://github.com/nsak-team/nsak>, 2026. Accessed: 2026-01-10.
- [9] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2):1–124, 2023.
- [10] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism, 2020.
- [11] Anthropic. Model context protocol. <https://www.anthropic.com/news/model-context-protocol>, 2024. Accessed: 2026-03-18.
- [12] Mohammed Mehedi Hasan, Hao Li, Emad Fallahzadeh, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E. Hassan. Model context protocol (mcp) at first glance: Studying the security and maintainability of mcp servers, 2025.
- [13] K Huang. Define ai agent. In Ken Huang, editor, *Agentic AI: Theories and Practices*. 2025.

- [14] J Huang K Huang. Define ai agent. In Ken Huang, editor, *Agentic AI: Theories and Practices*. 2025.
- [15] K Jackson C Hughes J Huang, K Huang. Ai agent safety and security considerations. In Ken Huang, editor, *Agentic AI: Theories and Practices*. 2025.
- [16] Risky Business Feature Podcast. ...mcp is dead. <https://risky.biz/>, 2026. Podcast episode , Accessed: 2026-03-18.
- [17] Ethan Michalak and Mark Perry. Connecting llms to caldera with the mcp plugin. <https://medium.com/@mitrecaldera/connecting-llms-to-caldera-with-the-mcp-plugin-da1450254827>, 2025. MITRE Caldera Blog, Accessed: 2026-03-16.

## List of Figures

## List of Tables

## List of Listings

## Glossary

This document is incomplete. The external file associated with the glossary ‘main’ (which should be called `thesis.gls`) hasn’t been created.

Check the contents of the file `thesis.gls`. If it’s empty, that means you haven’t indexed any of your entries in this glossary (using commands like `\gls` or `\glsadd`) so this list can’t be generated. If the file isn’t empty, the document build process hasn’t been completed.

If you don’t want this glossary, add `nomain` to your package option list when you load `glossaries-extra.sty`. For example:

```
\usepackage[nomain]{glossaries-extra}
```

Try one of the following:

- ▶ Add `automake` to your package option list when you load `glossaries-extra.sty`. For example:

```
\usepackage[automake]{glossaries-extra}
```

- ▶ Run the external (Lua) application:

```
makeglossaries-lite.lua "thesis"
```

- ▶ Run the external (Perl) application:

```
makeglossaries "thesis"
```

Then rerun  $\text{\LaTeX}$  on this document.

This message will be removed once the problem has been fixed.

# A Project Management

## A.1 Stakeholders

Hj. Wenger: Instructor and Network Specialist

Urs Keller: Expert and Entrepreneur

## A.2 Agile Management Process

The project followed an interactive development approach using Gitlab for issue tracking and planning. We defined our project management in a Scrum-like manner, but we individualized the process according to the projects scope and our needs. Work items where organized into epics, milestones and issues. Each iteration lasts two weeks. We decided to hold the following Scrum ceremonies after each sprint:

| <b>Sprint Planning</b>             | <b>Sprint Review</b>        | <b>Sprint Retro</b>     | <b>Checkpoint</b>        |
|------------------------------------|-----------------------------|-------------------------|--------------------------|
| Select milestone with sprint goal  | Close open tickets          | Reflect on sprint       | Present sprint results   |
| Check issues for DoR and obstacles | Gather team feedback        | Identify improvements   | Get stakeholder feedback |
| Ensure issues are clear            | Validate delivered features | Discuss what went well  | Define next tasks        |
| Create sprint backlog              | Review progress             | Discuss issues/problems | Discuss issues/problems  |

Table A.1: Key Scrum Meetings

The workflow used for ticket tracking consisted of the following stages:

### A.2.1 Issue Lifecycle:

| Buckets | Description                                                           |
|---------|-----------------------------------------------------------------------|
| Backlog | List of all tasks and features planned for the project.               |
| Todo    | Tasks selected for the current sprint that are ready to be worked on. |
| Q&A     | Tasks waiting for clarification, review, or testing.                  |
| Done    | Completed tasks that meet the defined acceptance criteria.            |

Table A.2: Issue Board

### A.2.2 Definition of Ready (DoR)

| Category   | Criteria                                                                                                                                                                     |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Clarity    | <ul style="list-style-type: none"><li>▶ Clear and precise title</li><li>▶ Objective or problem described in two to three sentences</li><li>▶ Scope clearly defined</li></ul> |
| Scope      | <ul style="list-style-type: none"><li>▶ Affected module specified (drill, scenario, core, container, frontend)</li><li>▶ External dependencies identified</li></ul>          |
| Validation | <ul style="list-style-type: none"><li>▶ Testable acceptance criteria defined</li><li>▶ Expected behavior clearly specified</li></ul>                                         |

Table A.3: Definition of Ready Criteria for Tickets